

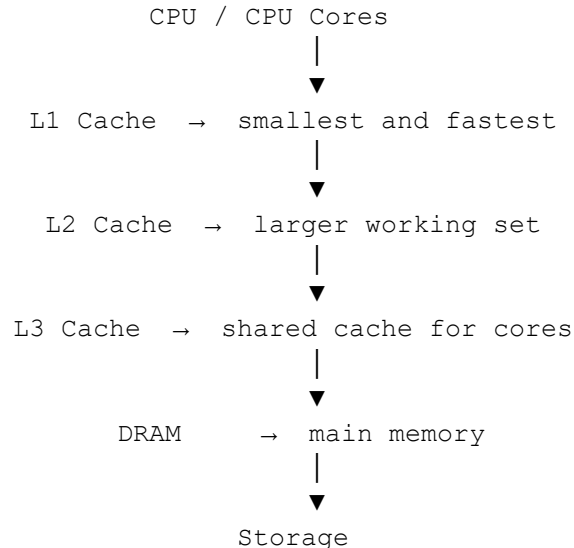
## 1. Smartphone Performance Optimization

A mobile manufacturer observes lag while switching between applications. The problem is mainly associated with repeated movement of application instructions and data through the memory hierarchy. The objective is to reduce memory-access latency and improve throughput by using L1, L2, L3 cache and DRAM effectively.

### Problem Analysis

- Application switching requires the processor to access instructions, application state, operating-system data and frequently used working data.
- If the required data is found in a cache, access is much faster than accessing DRAM.
- Frequent cache misses increase DRAM accesses and can cause processor stalls, producing visible switching delays.
- The redesigned hierarchy should keep the most frequently used data close to the processor while providing sufficient main-memory capacity.

### Proposed Memory Hierarchy



### Role of Each Level

| Memory   | Role in smartphone                               | Performance objective                     |
|----------|--------------------------------------------------|-------------------------------------------|
| L1 Cache | Store immediately required instructions and data | Minimize processor access latency         |
| L2 Cache | Hold a larger working set than L1                | Reduce L1 miss penalty                    |
| L3 Cache | Shared cache for data used by multiple cores     | Reduce DRAM traffic                       |
| DRAM     | Main memory for active applications              | Provide sufficient capacity and bandwidth |

### Recommended Improvements

- Use an appropriately sized L1 cache for highly time-critical instructions and data.
- Increase the effective L2/L3 cache capacity where workload analysis shows frequent cache misses.
- Use efficient cache replacement and prefetching so predictable application data reaches the cache before it is needed.

- Use high-speed DRAM with adequate bandwidth for multitasking workloads.
- Prioritize memory locality so frequently reused application data remains in the faster levels.

### Performance Analysis

The expected improvement is a reduction in average memory-access latency because a greater fraction of accesses can be served by L1, L2 or L3 cache. Reduced DRAM traffic also improves processor utilization and overall throughput. The design directly targets the reported app-switching lag.

### Conclusion

A balanced L1–L2–L3–DRAM hierarchy is suitable for smartphone multitasking. Keeping frequently accessed application data in the cache hierarchy while providing sufficient DRAM capacity reduces latency and improves responsiveness.

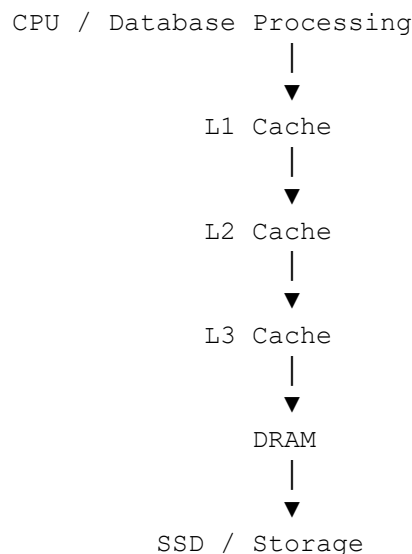
## 2. Cloud Server Memory Bottleneck

A cloud service provider experiences high latency in database queries. The solution must distinguish the role of cache hierarchy from virtual-memory paging and select memory technologies that reduce unnecessary slow accesses.

### Cache Hierarchy vs. Virtual Memory Paging

| Aspect          | Cache hierarchy                        | Virtual-memory paging                                       |
|-----------------|----------------------------------------|-------------------------------------------------------------|
| Purpose         | Reduce processor memory-access latency | Provide a larger logical address space using pages          |
| Fast path       | L1/L2/L3 cache hits                    | DRAM access when page is resident                           |
| Slow event      | Cache miss                             | Page fault                                                  |
| Main benefit    | Exploits temporal and spatial locality | Allows programs to use memory beyond physical DRAM capacity |
| Database impact | Fast access to repeatedly used data    | Page faults can cause very high access latency              |

### Proposed Architecture



## Recommended Improvements

- Keep frequently accessed database indexes and working data in memory so they benefit from cache locality.
- Provide sufficient DRAM to reduce page faults for active database workloads.
- Use an efficient page-replacement policy and avoid unnecessary paging.
- Use faster storage as the backing store for virtual memory, while recognizing that storage remains slower than DRAM.
- Use L1/L2/L3 cache effectively to reduce repeated DRAM accesses by the processor.

## Analysis

A cache miss is normally less disruptive than a page fault because the missing cache line can be obtained from a lower memory level, whereas a page fault may require storage access. Therefore, the recommended strategy is to maximize useful cache and DRAM residency and minimize page faults. This improves database-query response time and server throughput.

## Conclusion

Cache hierarchy and virtual memory solve different problems and should be used together. A cloud database server benefits from a strong L1–L2–L3 cache hierarchy, sufficient DRAM, and minimized paging activity.

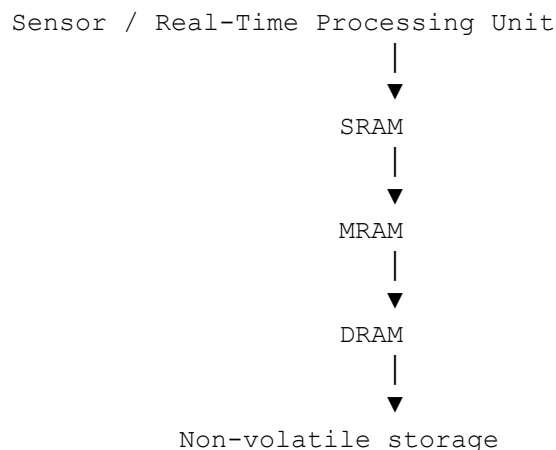
## 3. Autonomous Vehicle Data Processing

An autonomous vehicle must process sensor data in real time. The memory system must provide low latency, high bandwidth and reliable access for time-critical perception and control workloads.

### SRAM, DRAM and MRAM Comparison

| Technology | Strength                                          | Suitable use                                                           |
|------------|---------------------------------------------------|------------------------------------------------------------------------|
| SRAM       | Very low latency; high speed; volatile            | Time-critical buffers, cache and immediate sensor-processing data      |
| DRAM       | High capacity and good bandwidth; volatile        | Large sensor frames, maps and working datasets                         |
| MRAM       | Non-volatile and fast; retains data without power | Fast persistent state, configuration and selected frequently used data |

### Proposed Memory Hierarchy



## 2Design Strategy

- Use SRAM for the smallest and most latency-sensitive data path.
- Use DRAM for large sensor datasets and active processing buffers.
- Use MRAM where non-volatility and fast access are useful, such as selected system state and configuration information.
- Move data according to access frequency so critical information stays in the fastest level.
- Provide sufficient memory bandwidth to prevent the processor from waiting for large sensor-data transfers.

## Analysis

The hierarchy minimizes latency by placing time-critical information in SRAM and provides capacity through DRAM. MRAM adds a non-volatile fast-memory option. This combination is appropriate for real-time workloads because it balances speed, capacity and persistence.

## Conclusion

A layered SRAM–MRAM–DRAM organization provides a practical basis for autonomous-vehicle data processing. SRAM handles urgent data, DRAM handles large working sets, and MRAM supports fast non-volatile information storage.

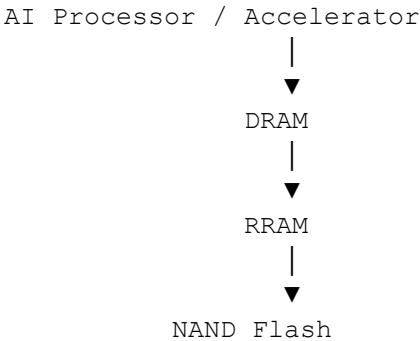
## 4. AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption. The memory system should separate active inference data from long-term model storage and use NAND Flash, DRAM and RRAM according to their characteristics.

## Technology Comparison

| Technology | Primary characteristic                                                    | Recommended role                                                                     |
|------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| NAND Flash | High density and non-volatile; suitable for persistent storage            | Store complete AI models, applications and datasets                                  |
| DRAM       | Fast volatile working memory with high capacity                           | Hold active model data, input data and intermediate results                          |
| RRAM       | Non-volatile memory technology suitable for fast, energy-efficient access | Store frequently used model data or selected model parameters close to the processor |

## Proposed Hierarchy



### Operating Strategy

- Store the complete model and application files in NAND Flash.
- Move the model or required model portions into faster memory before inference.
- Use DRAM for the active working set and intermediate inference data.
- Use RRAM for frequently reused non-volatile model data where the device architecture supports it.
- Reduce unnecessary movement between storage and working memory to lower energy consumption and improve model-loading time.

### Analysis

NAND Flash provides the required persistent capacity, while DRAM provides a fast working area for active inference. RRAM can complement these memories where fast non-volatile access is beneficial. The proposed hierarchy therefore balances model-storage capacity, access speed and power efficiency.

### Conclusion

An edge AI device should use NAND Flash for persistent storage, DRAM for active inference and RRAM for suitable frequently accessed non-volatile data. This hierarchy supports faster model loading and efficient edge inference.

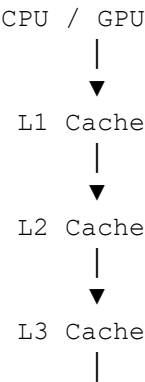
## 5. High-Performance Gaming Console

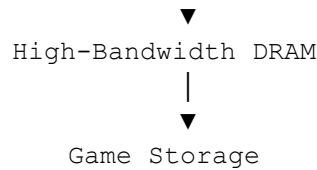
A gaming console suffers frame drops during large scene loading. The memory hierarchy must reduce processor stalls and provide enough bandwidth to transfer large game assets such as textures, maps and models.

### L3 Cache vs. DRAM

| Aspect           | L3 Cache                                 | DRAM                                               |
|------------------|------------------------------------------|----------------------------------------------------|
| Location         | Close to processor and commonly shared   | Main memory                                        |
| Capacity         | Smaller                                  | Much larger                                        |
| Latency          | Lower than DRAM                          | Higher than cache                                  |
| Role             | Keep frequently reused data close to CPU | Hold large active game datasets                    |
| Effect on gaming | Reduces repeated main-memory accesses    | Provides capacity and high data-transfer bandwidth |

### Proposed Architecture





## Recommended Improvements

- Improve effective L3 cache utilization for frequently reused game-engine data.
- Use high-bandwidth DRAM to handle large textures, scene data and other assets that cannot fit in cache.
- Use prefetching for predictable data so assets can be available before the processor or GPU requests them.
- Improve data locality to increase cache-hit rates.
- Reduce unnecessary transfers between storage and DRAM during scene transitions.

## Performance Analysis

L3 cache reduces the frequency of DRAM accesses for reusable data, while high-bandwidth DRAM provides the capacity and transfer rate needed for large scenes. Better locality and prefetching can reduce stalls during scene loading. The expected result is smoother rendering, fewer frame drops and improved overall throughput.

## Conclusion

The best gaming-console design is not to choose L3 cache instead of DRAM, but to use them together. L3 cache serves latency-sensitive reusable data, while high-bandwidth DRAM supports the large working set required by modern games.

## Overall Comparison and Design Summary

| Application        | Critical requirement                     | Recommended hierarchy                        |
|--------------------|------------------------------------------|----------------------------------------------|
| Smartphone         | Low app-switching latency                | L1 → L2 → L3 → DRAM                          |
| Cloud database     | Low query latency and fewer page faults  | L1 → L2 → L3 → DRAM → SSD                    |
| Autonomous vehicle | Real-time low latency and high bandwidth | SRAM → MRAM → DRAM                           |
| Edge AI            | Fast model loading and low power         | DRAM + RRAM + NAND Flash                     |
| Gaming console     | High bandwidth and low rendering stalls  | L1 → L2 → L3 → High-bandwidth DRAM → Storage |